

Practical -05

Code:

```
bharath_sai_reddy@LAPTOP-1  GNU nano 7.2  producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define N 100000

int main()
{
    int pipefd[2];

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        close(pipefd[1]);

        int sum = 0;
        int value;
```

```
int sum = 0;
int value;
int count = 0;

struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);

while (read(pipefd[0], &value, sizeof(value)) > 0)
{
    sum += value;
    count++;
}

clock_gettime(CLOCK_MONOTONIC, &end);

double time_taken =
    (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1e9;

close(pipefd[0]);

printf("Consumer: Received %d integers\n", count);
printf("Consumer: Sum of received data = %d\n", sum);
printf("Communication Time: %.6f seconds\n", time_taken);

if (time_taken > 0)
{
    printf("Communication Throughput: %.2f MB/s\n",
           (N * sizeof(int)) / (time_taken * 1024 * 1024));
}

wait(NULL);
```

```
bharath_sai_reddy@LAPTOP-1  ×  +  ▾
GNU nano 7.2 producer_consumer.c
}
else
{
    close(pipefd[0]);

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 1; i <= N; i++)
    {
        write(pipefd[1], &i, sizeof(i));
    }

    close(pipefd[1]);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Producer: Generated %d integers\n", N);
    printf("Producer: Sent %zu bytes through pipe\n",
        N * sizeof(int));
    printf("Producer Time: %.6f seconds\n", time_taken);

    wait(NULL);
}

return 0;
}
```

Output:

```
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/Practical-05$ ./producer_consumer
Producer: Generated 100000 integers
Producer: Sent 400000 bytes through pipe
Producer Time: 0.052864 seconds
Consumer: Received 100000 integers
Consumer: Sum of received data = 705082704
Communication Time: 0.055948 seconds
Communication Throughput: 6.82 MB/s
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/Practical-05$ |
```

Code:

```
GNU nano 7.2 ls_grep_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t child1 = fork();

    if (child1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (child1 == 0)
    {
        // First child: execute ls -l
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", (char *)NULL);

        perror("execlp ls");
        exit(1);
    }

    pid_t child2 = fork();

    if (child2 < 0)
    {
        perror("fork");
    }
}
```

```
bharath_sai_reddy@LAPTOP-1  ×  +  ∨
GNU nano 7.2 ls_grep_pipe.c
    close(pipefd[1]);

    execlp("ls", "ls", "-l", (char *)NULL);

    perror("execlp ls");
    exit(1);
}

pid_t child2 = fork();

if (child2 < 0)
{
    perror("fork");
    return 1;
}

if (child2 == 0)
{
    // Second child: execute grep ".c"
    close(pipefd[1]);

    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);

    execlp("grep", "grep", ".c", (char *)NULL);

    perror("execlp grep");
    exit(1);
}

// Parent process
close(pipefd[0]);
close(pipefd[1]);

waitpid(child1, NULL, 0);
waitpid(child2, NULL, 0);

printf("\nCommand executed successfully: ls -l | grep \".c\"\n");

return 0;
}
```

Output:

```
bharath_sai_reddy@LAPTOP-1711DF88:~/90128_OSSP/2520090128_Practical/Practical-05$ ./ls_grep_pipe
-rw-r--r-- 1 bharath_sai_reddy bharath_sai_reddy 1151 Aug 22 09:44 ls_grep_pipe.c
-rwxr-xr-x 1 bharath_sai_reddy bharath_sai_reddy 16368 Aug 22 09:42 producer_consumer
-rw-r--r-- 1 bharath_sai_reddy bharath_sai_reddy 1937 Aug 22 09:42 producer_consumer.c

Command executed successfully: ls -l | grep ".c"
```